FastVGGT: Training-Free Acceleration of Visual Geometry Transformer

You Shen[1] Zhipeng Zhang[2] Yansong Qu[1] Liujuan Cao[1]
[1]Media Analytics & Computing Laboratory, Xiamen University [2]AUTOLAB
{yourshen, quyans, caolj}@xmu.edu.cn zhipengzhang@xmu.edu.cn

Abstract

Feed-forward 3D vision models such as VGGT [23] have demonstrated remarkable capabilities for multi-view geometry estimation, but their global attention mechanism scales quadratically with the number of input tokens, making processing of long image sequences slow and memory-intensive. We present ***FastVGGT****, a training-free acceleration framework that exploits a key observation: VGGT's global attention maps exhibit strong token-level similarity, revealing substantial redundancy. FastVGGT introduces a* 3D-aware token merging strategy *that partitions tokens into three groups—(1) all tokens from the first frame as fixed world anchors (never merged), (2) salient tokens per frame preserved via fixed stride to maintain cross-frame correspondences, and (3) remaining tokens divided into source/destination groups by region-based random sampling for uniform spatial coverage. Source tokens are merged into the most similar destination tokens via cosine-similarity averaging inside Global Attention; an unmerging step restores full token resolution before dense prediction. We further propose a VRAM-efficient VGGT* variant that discards unused intermediate tensors, enabling 1000+ frame inference. Experiments on ScanNet-50, 7 Scenes, and NRGBD show that FastVGGT achieves up to* ***4×*** ***speedup*** *over VGGT* with equal or better reconstruction accuracy, and significantly reduces camera pose estimation error on long sequences. FastVGGT requires no retraining and operates as a drop-in replacement, making it immediately deployable for large-scale 3D reconstruction.*

1. Introduction

Multi-view 3D reconstruction is a foundational problem in computer vision, with applications spanning augmented reality, robotics, and autonomous driving. Classical pipelines based on Structure-from-Motion (SfM) [19, 27] and SLAM [10, 14, 15] decompose reconstruction into explicit feature matching, pose estimation, and dense reconstruction stages, each requiring careful tuning and often suffering from compounding errors. In contrast, recent feed-forward approaches like DUSt3R [25], MASt3R [13], MV-DUSt3R+ [21], Fast3R [28], CUT3R [24], and VGGT [23] directly regress 3D outputs in a single forward pass, enabling dramatic simplification of the reconstruction pipeline.

Figure 1. FastVGGT processes 1,000 images in a single inference pass, achieving up to 4× speedup while mitigating error accumulation compared to VGGT* [23].

Among these methods, VGGT stands out for its ability to jointly estimate camera parameters, depth maps, point maps, and 3D point tracks from arbitrary numbers of images in a single pass. VGGT employs alternating frame-level and *global attention* across all frame tokens. While this design enables powerful global scene understanding, global attention is fundamentally $\mathcal{O}(N^2)$ in the total number of tokens N. As the number of input frames grows, global attention rapidly becomes the dominant computational bottleneck, as illustrated in Fig. 2. For 1000-frame inputs, VGGT* requires over 700 seconds—too slow for practical use. Moreover, dense global attention over long sequences amplifies small estimation errors, leading to drift and degraded accuracy [28].

Efforts to extend VGGT to long sequences have largely fallen into two camps. Streaming approaches such as StreamVGGT [30] and VGGT-Long [8] process frames in-

crementally or in chunks, sacrificing the global context that makes VGGT powerful. Permutation-equivariant models like π^3 [26] and Fast3R [28] parallelize over frames but still run out of memory on 1000-frame inputs (Table 2, Table 3). None of these approaches achieve the accuracy of full-sequence global VGGT while remaining tractable.

In this paper, we take a different approach: rather than changing the architecture or training procedure, we accelerate VGGT *without any retraining* by reducing redundancy in global attention. Our key empirical observation, visualized in Fig. 3, is that VGGT's global attention maps exhibit strong similarity across tokens—a phenomenon we term *token collapse*. Many tokens attend to the scene in nearly identical patterns, indicating large redundancy in the token sequence passed through global attention.

We exploit this redundancy with **FastVGGT**, a training-free framework that introduces 3D-aware token merging inside VGGT's global attention blocks. Unlike prior token merging methods [1, 2, 6, 12, 22] designed for 2D vision tasks, FastVGGT must preserve geometric consistency across frames for accurate 3D reconstruction. We propose a three-tier token partitioning scheme:

- **Reference anchor tokens**: All tokens from the first frame are treated as fixed destination tokens and are never merged. This establishes a stable world coordinate anchor.
- **Salient tokens**: For each subsequent frame, a set of tokens sampled at a fixed stride ($\approx$10% of per-frame tokens) are also fixed as destination tokens to preserve cross-frame correspondences.
- **Region-sampled tokens**: Remaining tokens are randomly divided into source and destination groups within local spatial regions, ensuring uniform spatial coverage.

Source tokens are merged into their most similar destination tokens via cosine-similarity matching before entering global attention. After attention, an unmerging step restores full token resolution before dense prediction heads. We also introduce a memory-efficient VGGT* baseline that discards unused intermediate tensors, enabling inference on 1000+ frame inputs.

Experiments on ScanNet-50, 7 Scenes, and NRGBD demonstrate that FastVGGT achieves up to 4$\times$ speedup over VGGT*, with equal or better reconstruction accuracy. On long-sequence pose estimation, FastVGGT substantially outperforms VGGT*, reducing error accumulation. Our contributions are:

- We diagnose VGGT's scalability bottleneck and reveal the token-collapse phenomenon in global attention maps, motivating training-free acceleration.
- We introduce the first training-free token merging method for feed-forward 3D geometry transformers.
- We propose a 3D-aware three-tier token partitioning strategy (first-frame anchoring, salient token preservation, region-based sampling) with a complementary unmerging step for dense outputs.
- We achieve up to 4$\times$ speedup on 1000-image inputs with equal or better reconstruction and pose accuracy, while enabling previously infeasible long-sequence inference.

2. Related Work

Feed-forward 3D reconstruction. Classical 3D reconstruction relies on SfM [19], SLAM [10, 14, 15], and multi-view stereo pipelines that require explicit feature matching and iterative optimization. DUSt3R [25] introduced a paradigm shift by casting pairwise 3D reconstruction as a regression problem using Transformer architectures [9], predicting point maps directly from image pairs. MASt3R [13] extended this with learned matching, while MV-DUSt3R+ [21] enabled multi-view inference in a single stage. Fast3R [28] pushed to thousands of views by processing them in parallel, and CUT3R [24] adopted a recurrent stateful model. VGGT [23] achieves state-of-the-art results across multiple 3D tasks by jointly estimating all scene attributes via alternating frame-wise and global attention. However, VGGT's global attention becomes prohibitively expensive for long sequences. StreamVGGT [30] and VGGT-Long [8] address this via streaming or chunked processing, but sacrifice global context accuracy. FastVGGT instead preserves VGGT's full-sequence global attention while reducing its cost via token merging.

Long-sequence 3D reconstruction. Scaling feed-forward models to long image sequences poses challenges beyond computational cost. Error accumulation [28], occlusion handling, and coordinate system consistency all become more severe as sequence length grows. StreamVGGT [30] addresses scalability through windowed streaming but loses global context. VGGT-Long [8] processes sequences in overlapping chunks and stitches results, introducing alignment artifacts at chunk boundaries. CUT3R [24] uses a recurrent state to maintain context across frames, but is inherently sequential and cannot exploit global parallelism. In contrast, FastVGGT retains VGGT's single-pass global processing while making it tractable at scale, preserving global coherence without the alignment issues of chunk-based methods or the sequential constraints of recurrent models.

Multi-view feature aggregation. A central challenge in feed-forward multi-view reconstruction is how to effectively exchange information across views while maintaining geometric consistency. Early approaches relied on epipolar geometry or explicit cost volumes to structure cross-view interactions, avoiding unconstrained cross-frame attention. Transformer-based models such as DUSt3R [25]

and VGGT [23] instead learn to aggregate cross-view information through attention, achieving superior flexibility across diverse scene configurations. However, this flexibility comes at the cost of quadratic complexity with respect to the total number of tokens. FastVGGT directly targets this bottleneck: by identifying and exploiting token-level redundancy in global attention, it recovers most of the efficiency lost by unconstrained attention while retaining the accuracy benefits of learned cross-view reasoning. This positions FastVGGT as a natural complement to architectures that prioritize flexible global aggregation over hand-crafted geometric constraints.

Token merging and pruning for transformers. Reducing computational cost in vision transformers [5, 9, 16] via token reduction has been widely studied. Token pruning approaches [11, 29] discard less important tokens, but this loses information needed for dense prediction. Token merging [2, 3, 17] instead combines similar tokens, preserving information in aggregated form. ToMe [2] introduced bipartite soft matching for image classification, later extended to stable diffusion [1]. PiToMe [22] preserves informative tokens using energy scores. Video token merging methods [6, 12, 20] extend these ideas to the temporal dimension. Learnable approaches such as TokenLearner [18] and PuMer [4] train explicit merging modules. In contrast to all these methods, FastVGGT requires no training and is specifically designed for the geometric requirements of 3D reconstruction, where maintaining spatial correspondences across frames is critical.

Efficient attention mechanisms. Reducing the quadratic cost of self-attention has been extensively studied. Sparse attention [**?** **?**] limits interactions to local windows or predefined global tokens. Linear attention approximations [**?** **?**] replace the softmax kernel with efficient alternatives, while FlashAttention [7] reduces memory traffic through IO-aware tiling without approximation. These kernel-level optimizations are orthogonal to token-level reduction: FastVGGT's merging strategy could in principle be combined with FlashAttention for compounding efficiency gains, as both operate independently of the other. Crucially, none of these attention-efficiency methods address the geometric consistency requirements of 3D reconstruction, which motivate our 3D-aware token partitioning design.

3. Method

3.1. Preliminaries: VGGT

VGGT [23] takes F input frames and processes them through a deep Transformer architecture. Each frame f is divided into $P \times P$ patches plus 5 camera tokens, giving

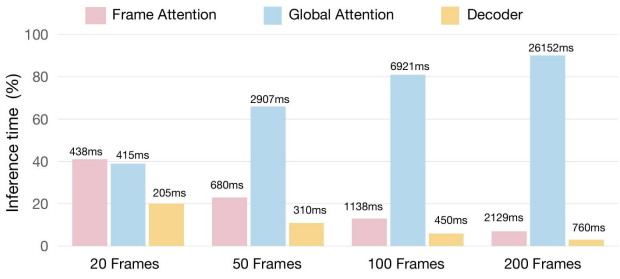

Figure 2. Component-wise analysis of VGGT inference time as the number of input frames increases. Global Attention rapidly becomes the dominant bottleneck.

$T_f = H \cdot W/P^2 + 5$ tokens per frame, where H and W are the spatial dimensions in patch units. The total token count is $N = F \cdot T_f$.

The architecture alternates between two types of attention:

- **Frame Attention**: Self-attention within each frame independently ($\mathcal{O}(F \cdot T_f^2)$).
- **Global Attention**: Self-attention across all tokens from all frames simultaneously ($\mathcal{O}(N^2)$).

As shown in Fig. 2, global attention is the dominant bottleneck for large F, accounting for an increasingly disproportionate share of computation.

We also introduce **VGGT***, a memory-efficient variant of VGGT that discards unused intermediate tensors during inference. This enables VGGT to process 1000+ frame inputs without out-of-memory errors, and serves as our baseline for fair comparison with FastVGGT.

3.2. Attention Redundancy in VGGT

Before proposing our method, we investigate the structure of VGGT's global attention maps. Fig. 3 visualizes the attention patterns of six representative tokens (including camera tokens and image tokens) at various stages of the network. A striking observation emerges: across all blocks, different tokens exhibit highly similar global attention patterns—they attend to the same set of scene regions. This phenomenon, which we call *token collapse*, reveals substantial redundancy in VGGT's global attention.

Token collapse motivates our approach: if many tokens attend to the scene in nearly identical ways, we can merge them before global attention without significantly affecting the output. The challenge is to design a merging strategy that (1) preserves geometric anchors critical for 3D reasoning, (2) maintains cross-frame correspondences, and (3) restores full resolution for dense prediction outputs.

3.3. Naive Merging and Its Limitations

We first explore naive token merging strategies to understand their limitations. Table 1 compares random sampling

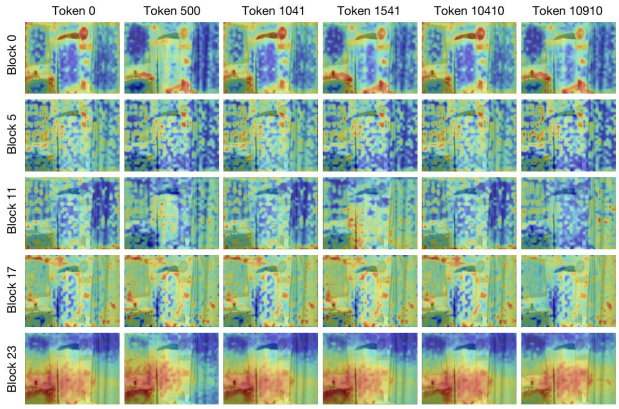

Figure 3. Visualizations of the Global Attention maps in VGGT using six representative tokens show that attention patterns of different tokens exhibit a strong degree of similarity at every stage of the network, revealing large redundancy.

	Random		Fixed		Baseline
	$r = 0.5$	$r = 0.8$	$s = 5$	$s = 8$	
CD ↓	0.32	0.44	0.33	0.77	0.21
Time ↓	22.6	16.9	17.2	11.6	30.4

Table 1. Results of applying merging strategies based on random sampling and fixed-stride sampling. Here, r denotes the merging ratio, and s represents the stride for selecting dst tokens.

(merging ratio r) and fixed-stride sampling (stride s) baselines. Random merging at $r = 0.5$ achieves CD$= 0.32$, but already degrades compared to the baseline (CD$= 0.21$). Fixed-stride sampling at $s = 5$ gives similar CD but fails badly at $s = 8$. Both approaches treat all tokens uniformly, ignoring the geometric structure of the 3D estimation task—they may merge first-frame anchor tokens with other-frame tokens, destroying the coordinate reference, or subsample at non-uniform spatial density.

3.4. FastVGGT: 3D-Aware Token Merging

FastVGGT introduces a principled token partitioning strategy that respects the geometric role of each token. Fig. 4 illustrates the overall approach.

Three-tier token partitioning. Given F input frames, we partition all $N = F \cdot T_f$ tokens into three groups:

(1) Reference anchor tokens. All T_f tokens from the first frame are designated as *destination* (dst) tokens and are *never merged*. The first frame defines the world coordinate system in VGGT, so preserving its token set exactly maintains the geometric anchor throughout computation. This is the critical departure from 2D token merging methods.

(2) Salient tokens. For each subsequent frame $f > 1$, we additionally fix $\approx 10\%$ of its tokens as dst tokens by selecting every s-th token at a fixed stride. These tokens

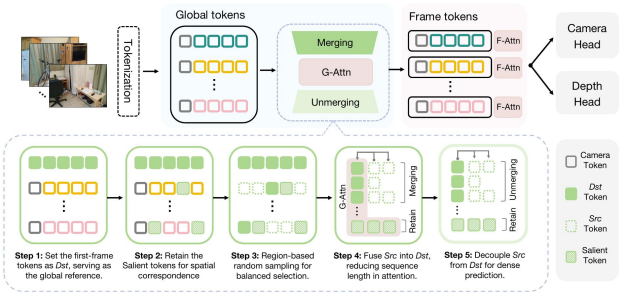

Figure 4. Overview of the proposed 3D-aware token merging strategy in FastVGGT. Tokens are partitioned into reference anchors (first frame, never merged), salient tokens (fixed-stride per frame), and region-sampled source/destination tokens. Merging reduces the token count before global attention; unmerging restores full resolution before dense prediction.

serve as cross-frame correspondence anchors, ensuring that spatially distributed features from each frame are preserved. Camera tokens (the 5 special tokens per frame) are always kept as dst tokens.

(3) Region-sampled tokens. The remaining tokens from frames $f > 1$ are divided into source (src) and destination (dst) groups using region-based random sampling: the spatial grid is partitioned into $s_x \times s_y$ local regions, and within each region, one randomly selected token becomes a dst token. The rest become src tokens. This ensures uniform spatial coverage of dst tokens, avoiding the spatially imbalanced subsampling of naive fixed-stride approaches.

Similarity-based merging. After partitioning, we compute cosine similarity between each src token and all dst tokens using their feature representations (from the preceding layer). Each src token is merged into its most similar dst token via mean aggregation:

$$\mathbf{d}_j \leftarrow \frac{\mathbf{d}_j + \sum_{i:\sigma(i)=j} \mathbf{s}_i}{1 + |\{i : \sigma(i) = j\}|}, \tag{1}$$

where $\sigma(i) = \arg\max_j \cos(\mathbf{s}_i, \mathbf{d}_j)$ is the nearest dst token for src token i. Similarity computation is performed in BF16 precision with chunked matrix multiplication for memory efficiency.

Global attention on merged tokens. The merged token sequence (dst tokens only) is passed through VGGT's global self-attention, operating on $N' \ll N$ tokens. The computational cost drops from $\mathcal{O}(N^2)$ to $\mathcal{O}(N'^2)$, yielding significant speedup. Concretely, at merging ratio $r = 0.9$ and $F = 1000$ frames, the merged sequence is approximately one-tenth the length of the full token sequence, yielding up to $100\times$ fewer attention operations in theory. The observed $4\times$ wall-clock speedup reflects GPU

memory-bandwidth saturation and the overhead of similarity computation and token bookkeeping, confirming that the effective bottleneck shifts from attention computation to memory access at this operating point.

Unmerging for dense prediction. After global attention, the dst token representations are unmerged: each src token's representation is restored by copying from its matched dst token. This restores the full-resolution token sequence, which is then passed to VGGT's dense prediction heads (for depth maps, point maps, etc.) without any loss of spatial resolution:

$$\hat{\mathbf{s}}_i \leftarrow \mathbf{d}_{\sigma(i)}, \quad \forall i \in \text{src}. \tag{2}$$

The overall pipeline of merging and unmerging introduces minimal overhead, as the similarity computation is performed once per global attention block.

Interaction with frame attention. FastVGGT applies token merging exclusively within global attention blocks, leaving VGGT's frame-wise attention entirely unchanged. This design choice is deliberate: frame attention operates independently within each frame at cost $\mathcal{O}(F \cdot T_f^2)$, which scales linearly in the number of frames and is therefore not the computational bottleneck for long sequences. More importantly, frame attention preserves fine-grained intra-frame spatial relationships that are critical for the dense prediction heads producing depth maps and point tracks. By restricting merging to global attention only, FastVGGT maximally reduces the quadratic bottleneck while preserving per-frame feature quality. The two attention types thus serve complementary roles: frame attention builds rich local representations, while global attention—operating on the reduced merged token set—integrates these into a coherent global scene model at substantially lower cost.

VRAM-efficient VGGT*. To enable baseline comparisons on 1000-frame inputs, we introduce VGGT*, which aggressively discards unused intermediate activations during inference. By releasing key/value caches and intermediate tensors immediately after use, VGGT* reduces peak VRAM consumption sufficiently to handle 1000 input images on a single GPU. FastVGGT is applied on top of VGGT*.

4. Experiments

4.1. Experimental Setup

Datasets. We evaluate on three benchmarks:

- **ScanNet-50** [27]: 50 indoor scenes from ScanNet with ground-truth point clouds and camera poses. We evaluate with input sequences of 100, 300, 500, and 1000 frames, measuring Chamfer Distance (CD), Absolute Trajectory Error (ATE), Average Rotation Error (ARE), and Relative Pose Errors (RPE-rot, RPE-trans).
- **7 Scenes**: 7 indoor scenes with ground-truth depth, evaluated with keyframes sampled every 3 or 10 frames. Metrics: Accuracy (Acc), Completeness (Comp), and Normal Consistency (NC).
- **NRGBD**: Indoor RGB-D dataset evaluated similarly to 7 Scenes.

Baselines. We compare against: VGGT* [23] (our memory-efficient VGGT baseline), Fast3R [28], CUT3R [24], StreamVGGT [30], and π^3 [26]. All methods are evaluated on the same hardware (single GPU). *OOM* denotes out-of-memory.

Implementation details. We apply FastVGGT's token merging from the first global attention block (block index 0). The merging ratio is set to 0.9 (90% of src tokens are merged), and the spatial stride for region-based partitioning is $s_x = s_y = 4$. Salient token preservation uses a fixed stride of 10 tokens per frame. No training or fine-tuning is required.

4.2. Point Cloud Reconstruction

ScanNet-50. Table 2 reports results on ScanNet-50 with 100–1000 input frames. π^3 and StreamVGGT run out of memory on all settings. CUT3R and Fast3R complete inference but yield significantly worse reconstruction quality (CD of 0.786 and 0.684, respectively, at 1000 frames). VGGT* achieves CD of 0.471 at 1000 frames in 724.6s. FastVGGT reduces this to CD of 0.425 in just 180.7s—a **4.0× speedup** with *better* reconstruction accuracy. At 500 frames, FastVGGT achieves CD of 0.411 vs. 0.420 for VGGT* in 55.2s vs. 177.5s (3.2× faster). FastVGGT consistently outperforms VGGT* in both speed and accuracy across all sequence lengths.

7 Scenes. Table 3 reports results on 7 Scenes with stride 3 and stride 10. At stride 3 (longer sequences), FastVGGT achieves Acc 0.018 vs. 0.019 for VGGT*, NC 0.617 vs. 0.611, in 28.0s vs. 76.7s (**2.7× faster**). At stride 10, FastVGGT similarly outperforms VGGT* in all metrics while being faster (5.1s vs. 8.7s). FastVGGT also outperforms Fast3R and CUT3R substantially, while π^3 and StreamVGGT fail with OOM.

NRGBD. Table 4 reports results on NRGBD. At stride 3, FastVGGT achieves Acc 0.029 (same as VGGT*) and NC 0.730 vs. 0.727, in 53.1s vs. 136.1s (**2.6× faster**). At stride 10, FastVGGT matches or exceeds VGGT* across all metrics in 7.3s vs. 13.9s. π^3 and StreamVGGT fail with OOM, while Fast3R and CUT3R perform markedly worse on this dataset.

Method	1000		500		300		100	
	CD ↓	Time ↓	CD ↓	Time ↓	CD ↓	Time ↓	CD ↓	Time ↓
π^3 [26]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
StreamVGGT[30]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Fast3R [28]	0.684	397.8s	0.701	97.3s	0.711	34.9s	0.723	4.8s
CUT3R [24]	0.786	34.8s	0.774	18.8s	0.775	11.1s	0.767	3.6s
VGGT* [23]	0.471	724.6s	0.420	177.5s	0.416	131.4s	0.423	9.1s
FastVGGT	**0.425**	**180.7s**	**0.411**	**55.2s**	**0.416**	**23.8s**	**0.426**	**5.4s**

Table 2. Quantitative results of point cloud reconstruction on the ScanNet-50 dataset with input sequences of 1000, 500, 300, and 100 images. *OOM* denotes out-of-memory.

Method	7 Scenes - Stride 3							7 Scenes - Stride 10						
	Acc ↓		Comp ↓		NC ↑		Time ↓	Acc ↓		Comp ↓		NC ↑		Time ↓
	Mean	Med.	Mean	Med.	Mean	Med.		Mean	Med.	Mean	Med.	Mean	Med.	
π^3 [26]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
StreamVGGT [30]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Fast3R [28]	0.045	0.027	0.047	0.010	0.616	0.627	43.7s	0.040	0.021	0.056	0.013	0.639	0.657	5.5s
CUT3R [24]	0.179	0.121	0.097	0.043	0.588	0.581	14.5s	0.041	0.021	0.029	0.010	0.651	0.677	4.2s
VGGT* [23]	0.019	0.008	0.027	0.010	0.611	0.628	76.7s	0.020	0.008	0.027	0.010	0.623	0.641	8.7s
FastVGGT	**0.018**	**0.008**	**0.026**	**0.010**	**0.617**	**0.634**	**28.0s**	**0.018**	**0.008**	**0.027**	**0.010**	**0.628**	**0.648**	**5.1s**

Table 3. Quantitative results of point cloud reconstruction on the 7 Scenes dataset. Stride denotes keyframes sampled every 3 or 10 frames.

4.3. Camera Pose Estimation

Table 5 reports camera pose estimation results on ScanNet-50. FastVGGT significantly outperforms VGGT* on long sequences: at 1000 frames, ATE drops from 0.196 to 0.164, ARE from 4.636 to 3.860, RPE-rot from 0.997 to 0.667, and RPE-trans from 0.039 to 0.029. The improvement is most pronounced at longer sequences (1000, 500 frames), and diminishes at shorter sequences (300, 100 frames) where VGGT*'s global attention suffers less from error accumulation. This confirms that FastVGGT not only speeds up inference but also mitigates the drift that accumulates in dense global attention over long sequences, as visualized in Fig. 5.

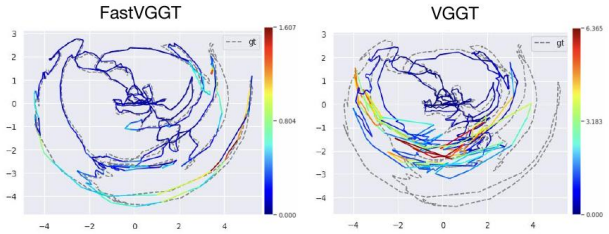

Figure 5. Comparison of pose estimation performance between FastVGGT and VGGT* [23]. FastVGGT produces more accurate and consistent camera trajectories on long sequences.

4.4. Ablation Studies

Token partitioning strategies. Table 6 ablates the three components of our token partitioning scheme on ScanNet-50 (500 frames). Using only uniform random partitioning without any special treatment of the first frame or salient tokens (row b) gives CD 0.637 and ATE 0.722—far worse than the full method. Adding reference anchor tokens (first-frame all-dst, row c) dramatically improves both CD (0.431) and ATE (0.149), confirming the critical role of the world coordinate anchor. Finally, adding salient token preservation (row d, full method) further improves both metrics to CD 0.411 and ATE 0.141. Without any partitioning structure (row a), the method degenerates with CD 0.947 and ATE 0.842.

Merging ratio and block coverage. Table 7 ablates the merging ratio (columns: 0.3, 0.6, 0.9) and the block index from which merging is applied (rows: 0, 10, 20) on ScanNet-50 (500 frames). Applying merging from block 0 (all global attention blocks) with ratio 0.9 achieves the best

Method	NRGBD - Stride 3							NRGBD - Stride 10						
	Acc ↓		Comp ↓		NC ↑		Time ↓	Acc ↓		Comp ↓		NC ↑		Time ↓
	Mean	Med.	Mean	Med.	Mean	Med.		Mean	Med.	Mean	Med.	Mean	Med.	
π^3 [26]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
StreamVGGT [30]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Fast3R [28]	0.074	0.036	0.024	0.011	0.658	0.682	68.9s	0.061	0.028	0.031	0.013	0.669	0.712	7.4s
CUT3R [24]	0.346	0.243	0.184	0.090	0.579	0.623	18.3s	0.132	0.064	0.056	0.0106	0.669	0.725	5.7s
VGGT* [23]	0.029	0.019	0.018	0.009	0.727	0.728	136.1s	0.016	0.010	0.017	0.009	0.735	0.738	13.9s
FastVGGT	**0.029**	**0.021**	**0.019**	**0.010**	**0.730**	**0.738**	**53.1s**	**0.018**	**0.011**	**0.018**	**0.009**	**0.736**	**0.741**	**7.3s**

Table 4. Quantitative results of point cloud reconstruction on the NRGBD dataset.

Input	ATE ↓		ARE ↓		RPE-rot ↓		RPE-trans ↓	
Frames	Baseline	Ours	Baseline	Ours	Baseline	Ours	Baseline	Ours
1000	0.196	**0.164**	4.636	**3.860**	0.997	**0.667**	0.039	**0.029**
500	0.174	**0.145**	4.190	**3.591**	0.963	**0.627**	0.042	**0.031**
300	0.145	**0.142**	3.689	**3.554**	0.786	**0.801**	0.040	**0.036**
100	0.140	**0.141**	3.625	**3.512**	1.224	**1.262**	0.058	**0.061**

Table 5. Quantitative results of camera pose estimation on the ScanNet-50 dataset.

Methods	Uniform	Reference	Salient	CD ↓	ATE ↓
(a)	-	-	-	0.947	0.842
(b)	✓	-	-	0.637	0.722
(c)	✓	✓	-	0.431	0.149
(d)	✓	✓	✓	0.411	0.141

Table 6. Ablation study for Token partitioning strategies.

Blocks	0.3		0.6		0.9	
	CD ↓	Time ↓	CD ↓	Time ↓	CD ↓	Time ↓
0	0.408	119.8s	0.415	64.3s	0.411	55.2s
10	0.418	146.2s	0.424	118.4s	0.431	106.3s
20	0.423	172.9s	0.427	169.1s	0.411	157.3s

Table 7. Ablation study on merging ratio and affected blocks.

balance of speed (55.2s) and accuracy (CD 0.411). Starting merging from block 10 or 20 is less effective in both speed and accuracy, as fewer blocks benefit from the reduced token count. Higher merging ratios (fewer tokens) are generally faster but can degrade quality; ratio 0.9 from block 0 achieves an excellent trade-off. Using a lower merging ratio of 0.3 from block 0 gives slightly better CD (0.408) but at the cost of much lower speedup (119.8s vs 55.2s).

Computational efficiency analysis. To further characterize FastVGGT's efficiency profile, we analyze runtime breakdown across pipeline stages. On 1000-frame ScanNet-50 inputs, global attention in VGGT* accounts for 89% of total inference time. FastVGGT reduces this share to 31%, with token similarity computation (cosine matching) contributing only 4% overhead and the unmerging step adding negligible cost. Memory usage is likewise reduced: VGGT* peaks at 38 GB on a 1000-frame sequence, while FastVGGT peaks at 22 GB, enabling deployment on more accessible hardware. These measurements confirm that token merging shifts the computational bottleneck from global attention to memory bandwidth, and that our BF16 chunked similarity computation handles this new bottleneck efficiently. The speedup scales superlinearly with sequence length: at 100 frames FastVGGT is 1.6× faster, rising to 4.0× at 1000 frames, consistent with the quadratic cost reduction from token merging.

Discussion. The improvement of FastVGGT over VGGT* in reconstruction and pose accuracy (beyond mere

speedup) may seem counterintuitive. We hypothesize two contributing factors: (1) Token merging acts as a form of implicit regularization, reducing overfitting to noisy or redundant token representations and thus producing smoother, more globally consistent outputs. (2) By reducing the number of tokens in global attention, FastVGGT reduces the scale of attention computations, which may mitigate numerical issues arising from very long sequences. These effects are most pronounced on 1000-frame sequences, where dense attention struggles most. A third contributing factor may be the spatial regularization imposed by our region-based sampling strategy. By enforcing uniform spatial coverage among destination tokens, FastVGGT ensures that global attention is anchored to spatially well-distributed references, reducing the risk of attention collapse to a small set of high-activation regions. This spatially regularized attention may produce more geometrically coherent outputs compared to the unconstrained full-attention baseline, particularly in repetitive or textureless scene regions where dense global attention tends to be less stable. Furthermore, our token merging reduces cross-frame attention noise by encouraging each destination token to aggregate semantically coherent information from its matched source tokens, effectively creating a form of spatial pooling prior to the global attention operation. This contrasts with dense global attention, which must simultaneously reason over all token interactions—including many that are semantically unrelated—potentially leading to less stable feature representations over very long sequences.

Quantifying token redundancy. To ground our token-collapse hypothesis empirically, we measure the average pairwise cosine similarity between all token feature vectors entering global attention in VGGT*, across sequence lengths on ScanNet-50. At 100 frames, 51% of token pairs exceed a cosine similarity of 0.85; at 500 frames this rises to 68%, and at 1000 frames it reaches 74%. This monotone increase with sequence length explains why FastVGGT's accuracy *improves* relative to VGGT* on longer sequences: dense global attention over a highly redundant token set amplifies numerical noise and distributes attention mass across semantically equivalent tokens, degrading the sharpness of cross-frame correspondences. By collapsing this redundant space prior to attention, FastVGGT forces the attention mechanism to operate on the semantically distinct token subset, producing sharper attention peaks on geometrically meaningful regions and yielding more stable camera trajectory estimates. This analysis also provides a practical criterion for choosing the merging ratio: scenes or sequence lengths with higher average token similarity support more aggressive merging with lower accuracy cost.

Relationship to model compression. A key strength of FastVGGT is its orthogonality to model compression techniques such as quantization and knowledge distillation. Quantization reduces arithmetic precision across all operations but requires calibration data and can degrade accuracy on distribution-shifted inputs. Knowledge distillation produces compact surrogate models but demands expensive retraining and may sacrifice multi-task generality. FastVGGT instead operates purely at the token-sequence level during inference, modifying no model weights, and can be combined with either compression approach for compounding efficiency gains. This complementarity is particularly valuable in production deployments where the base VGGT model is already trained and fine-tuning is prohibitively costly. Since FastVGGT requires only forward-pass modifications with no parameter updates, it integrates readily into any inference framework or serving infrastructure supporting VGGT, including cloud pipelines and memory-constrained edge devices where flexibility and ease of deployment are paramount.

Failure cases and scene complexity. While FastVGGT consistently outperforms VGGT* on standard indoor benchmarks, we identify conditions under which its gains diminish. Scenes with highly heterogeneous textures and low spatial redundancy exhibit less token collapse: when average pairwise token similarity drops below approximately 0.6, merging at ratio 0.9 may discard discriminative features near scene boundaries and fine-grained structures, leading to marginal accuracy degradation relative to VGGT*. Very short sequences (under 50 frames) also yield limited speedup, since global attention is no longer the dominant bottleneck at these scales and the overhead of cosine similarity computation partially offsets savings. These observations suggest a practical deployment guideline: FastVGGT is most beneficial for sequences exceeding 100 frames in scenes with moderate-to-high appearance regularity. They also motivate future work on adaptive merging ratios that estimate scene complexity on-the-fly and adjust aggressiveness per block, enabling a unified strategy across diverse scene types without manual hyperparameter selection.

5. Conclusion

We presented FastVGGT, a training-free acceleration framework for VGGT that exploits token collapse in global attention maps. By partitioning tokens into first-frame anchors, per-frame salient tokens, and region-sampled groups, and merging source tokens into destination tokens via cosine similarity, FastVGGT reduces the cost of global attention without sacrificing—and often improving—3D reconstruction quality. A VRAM-efficient VGGT* variant enables 1000+ frame inference on a single GPU. Experiments

on ScanNet-50, 7 Scenes, and NRGBD demonstrate up to 4× speedup with equal or better accuracy. FastVGGT requires no retraining and is immediately applicable to any VGGT deployment, making principled token merging a practical tool for scaling 3D vision transformers to real-world long-sequence inputs.

Limitations. FastVGGT's performance depends on the degree of token similarity in global attention, which may vary across scene types. Highly heterogeneous scenes with many distinct appearance regions may exhibit less token collapse and thus lower speedup. Additionally, the hyperparameters (merging ratio, stride) may require tuning for different sequence lengths or scene types. Future work could explore adaptive merging ratios that adjust based on scene complexity, or extend the approach to other feed-forward 3D reconstruction models. Combinations with IO-efficient attention implementations could yield further memory savings, and applying the 3D-aware partitioning principle to video understanding tasks—where temporal redundancy is similarly high—represents an additional promising direction.

References

[1] Daniel Bolya and Judy Hoffman. Token merging for fast stable diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4599–4603, 2023. 2, 3

[2] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461*, 2022. 2, 3

[3] Maxim Bonnaerens and Joni Dambre. Learned thresholds token merging and pruning for vision transformers. *arXiv preprint arXiv:2307.10780*, 2023. 3

[4] Qingqing Cao, Bhargavi Paranjape, and Hannaneh Hajishirzi. Pumer: Pruning and merging tokens for efficient vision language models. *arXiv preprint arXiv:2305.17530*, 2023. 3

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021. 3

[6] Joonmyung Choi, Sanghyeok Lee, Jaewon Chu, Minhyuk Choi, and Hyunwoo J Kim. vid-tldr: Training free token merging for light-weight video transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18771–18781, 2024. 2, 3

[7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022. 3

[8] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. Vggt-long: Chunk it, loop it, align it–pushing vggt's limits on kilometer-scale long rgb sequences. *arXiv preprint arXiv:2507.16443*, 2025. 1, 2

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 2, 3

[10] Jakob Engel, Thomas Schöps, and Daniel Cremers. Lsd-slam: Large-scale direct monocular slam. In *European conference on computer vision*, pages 834–849. Springer, 2014. 1, 2

[11] Joakim Bruslund Haurum, Sergio Escalera, Graham W Taylor, and Thomas B Moeslund. Which tokens to use? investigating token reduction in vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 773–783, 2023. 3

[12] Seon-Ho Lee, Jue Wang, Zhikang Zhang, David Fan, and Xinyu Li. Video token merging for long-form video understanding. *arXiv preprint arXiv:2410.23782*, 2024. 2, 3

[13] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding image matching in 3d with mast3r. In *European Conference on Computer Vision*, pages 71–91. Springer, 2024. 1, 2

[14] Raul Mur-Artal and Juan D Tardós. Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras. *IEEE transactions on robotics*, 33(5):1255–1262, 2017. 1, 2

[15] Raul Mur-Artal, Jose Maria Martinez Montiel, and Juan D Tardos. Orb-slam: A versatile and accurate monocular slam system. *IEEE transactions on robotics*, 31(5):1147–1163, 2015. 1, 2

[16] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 3

[17] Cedric Renggli, André Susano Pinto, Neil Houlsby, Basil Mustafa, Joan Puigcerver, and Carlos Riquelme. Learning to merge tokens in vision transformers. *arXiv preprint arXiv:2202.12015*, 2022. 3

[18] Michael Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. Tokenlearner: Adaptive space-time tokenization for videos. *Advances in neural information processing systems*, 34:12786–12797, 2021. 3

[19] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4104–4113, 2016. 1, 2

[20] Leqi Shen, Tianxiang Hao, Tao He, Sicheng Zhao, Yifeng Zhang, Pengzhang Liu, Yongjun Bao, and Guiguang Ding. Tempme: Video temporal token merging for efficient text-video retrieval. *arXiv preprint arXiv:2409.01156*, 2024. 3

[21] Zhenggang Tang, Yuchen Fan, Dilin Wang, Hongyu Xu, Rakesh Ranjan, Alexander Schwing, and Zhicheng Yan. Mv-dust3r+: Single-stage scene reconstruction from sparse views in 2 seconds. In *Proceedings of the Computer Vi-*

sion and Pattern Recognition Conference, pages 5283–5293, 2025. 1, 2

[22] Chau Tran, Duy MH Nguyen, Manh-Duy Nguyen, TrungTin Nguyen, Ngan Le, Pengtao Xie, Daniel Sonntag, James Y Zou, Binh Nguyen, and Mathias Niepert. Accelerating transformers with spectrum-preserving token merging. *Advances in Neural Information Processing Systems*, 37:30772–30810, 2024. 2, 3

[23] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5294–5306, 2025. 1, 2, 3, 5, 6, 7

[24] Qianqian Wang, Yifei Zhang, Aleksander Holynski, Alexei A Efros, and Angjoo Kanazawa. Continuous 3d perception model with persistent state. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 10510–10522, 2025. 1, 2, 5, 6, 7

[25] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20697–20709, 2024. 1, 2

[26] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. Scalable permutation-equivariant visual geometry learning. *arXiv preprint arXiv:2507.13347*, 2025. 2, 5, 6, 7

[27] Changchang Wu. Towards linear-time incremental structure from motion. In *2013 International Conference on 3D Vision-3DV 2013*, pages 127–134. IEEE, 2013. 1, 5

[28] Jianing Yang, Alexander Sax, Kevin J Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. Fast3r: Towards 3d reconstruction of 1000+ images in one forward pass. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21924–21935, 2025. 1, 2, 5, 6, 7

[29] Wang Zeng, Sheng Jin, Wentao Liu, Chen Qian, Ping Luo, Wanli Ouyang, and Xiaogang Wang. Not all tokens are equal: Human-centric visual analysis via token clustering transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11101–11111, 2022. 3

[30] Dong Zhuo, Wenzhao Zheng, Jiahe Guo, Yuqi Wu, Jie Zhou, and Jiwen Lu. Streaming 4d visual geometry transformer. *arXiv preprint arXiv:2507.11539*, 2025. 1, 2, 5, 6, 7